SE Experiment-3
(Batch-D/C4)

Team Details
Vidhaan Shah – UID (2024300212)
Meet Shah – UID (2024300226)
T.E. Computer Engineering – C

Aim:
To draw a Class diagram for a Pet Boarding and Grooming Management System.

Implementation:

Mentioned below is the chronological order that was followed to design a Class Diagram for our Pet Boarding and Grooming Management System. Initially, the problem description was re-visited to identify nouns and noun phrases. These were then reviewed to get rid of any ambiguity and the required classes were identified.

Relations between the different classes and how they interact with each other were then studied to get an idea about the overall structure of the class diagram. Finally, the class diagram was constructed by defining the attributes, operations, associations, generalization relationships and multiplicities between the identified classes.

Problem Description

A **Pet Boarding and Grooming Management System** needs to be designed to manage pets, pet owners, bookings, boarding services, grooming services, staff, resources and payments.

A **Pet Owner** can register and log into the system. The pet owner can add and manage details of one or more pets. The system stores important information about every pet, such as its name, species, breed, age and special requirements.

The pet owner can create bookings for different services provided by the facility. The system provides **Boarding Services** and **Grooming Services**. Boarding services maintain information such as the pet, start date, end date, status and total cost. Grooming services maintain details such as the pet, service type, date, time,

status and total cost.

A **Booking** is created for the requested service and can be updated or cancelled. Each booking may have an associated **Payment**. The system records the payment amount, payment method, payment status, transaction ID and payment date. Refunds can also be processed when required.

The system also manages **Check-In and Check-Out** records for pets. These records store the pet details, booking details, check-in date and time, check-out date and time and relevant notes.

The facility has **Staff** members who can log into the system, view their schedules and update their assigned schedules. An **Admin** manages the staff and users and can generate reports related to the overall functioning of the system.

The system maintains details of available **Resources**, such as rooms, kennels and grooming resources. Resource availability can be checked and updated. An **Inventory** is maintained to manage items, quantities and stock availability.

The system also generates **Notifications** for important events such as bookings, payments and reminders. Different types of reports can be generated for a specified period.

Following is a breakdown of various parts that were considered for designing the class diagram-

[1] Noun/Noun Phrases.

Pet Owner, Pet, User, Staff, Admin, Boarding Service, Grooming Service, Booking, Payment, Check-In/Check-Out, Notification, Schedule, Resource, Inventory and Report.

[2] Classes.

The following classes were identified:

- PetOwner
- Pet
- User
- Staff
- Admin
- BoardingService
- GroomingService
- Booking
- Payment
- CheckInOut
- Notification
- Schedule
- Resource
- Inventory
- Report

[3] Verb Phrases.

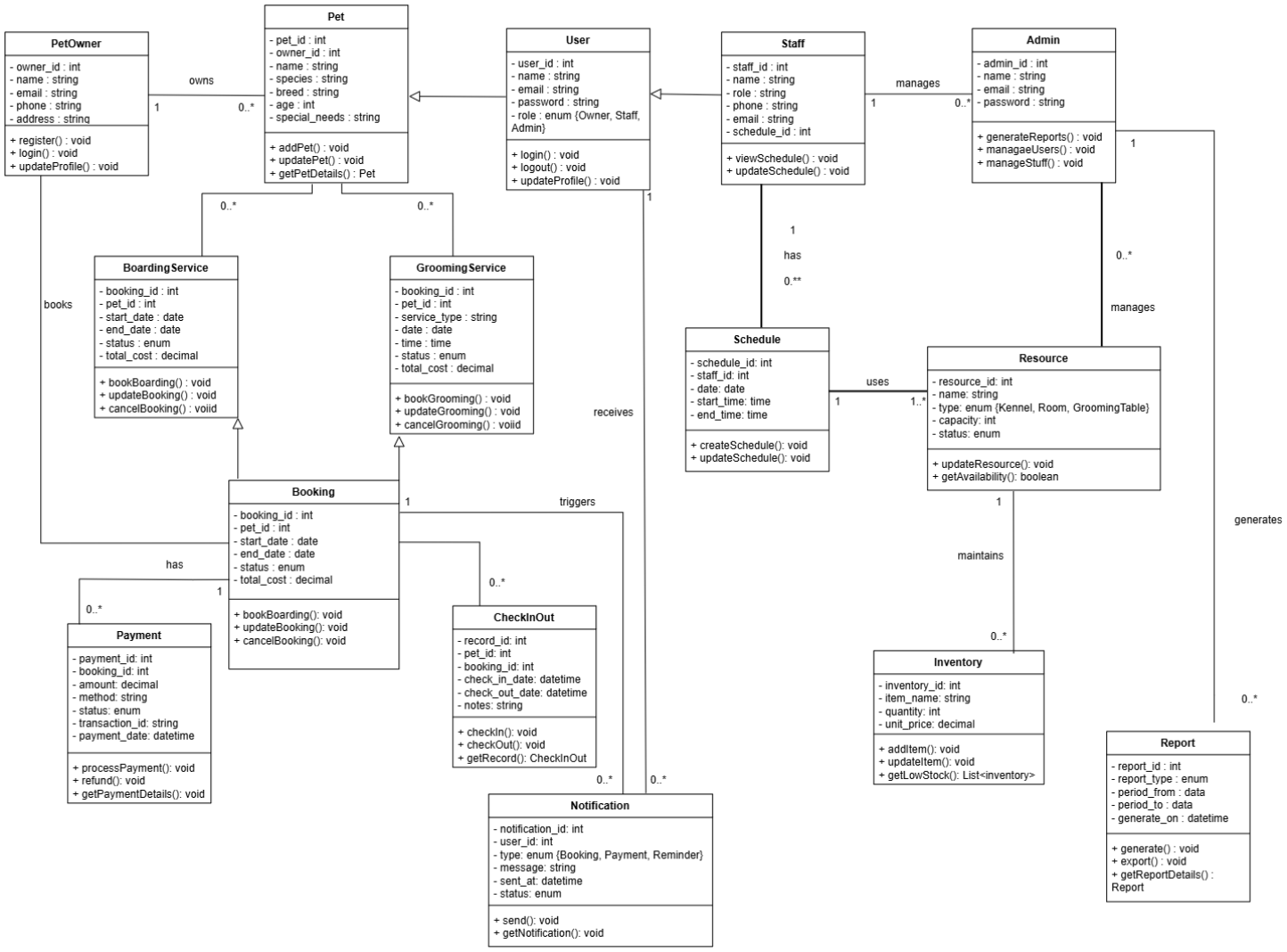
1. Pet Owner owns and manages Pets.
2. Pet Owner registers and logs into the system.
3. Pet Owner updates their profile.
4. A Pet has details such as name, species, breed, age and special needs.
5. A Pet Owner can create one or more Bookings for their Pets.
6. A Booking can be associated with Boarding Service or Grooming Service.
7. Boarding Service manages the boarding period and status of a Pet.
8. Grooming Service manages the type and timing of grooming requested for a Pet.

9. A Booking may have an associated Payment.
10. Payment processes payments and refunds.
11. A Booking may have Check-In and Check-Out details.
12. A Booking can generate Notifications.
13. Staff members view and update their Schedules.
14. Schedules use available Resources.
15. Resources are maintained through Inventory.
16. Admin manages Staff and Users.
17. Admin generates Reports.

[4] Relations.

1. A **Pet Owner** may own zero or more **Pets**, whereas each Pet belongs to one Pet Owner.
2. A **Pet** may have zero or more service bookings.
3. A **Booking** is associated with either a **Boarding Service** or a **Grooming Service**, depending on the service selected.
4. A **Booking** may have zero or more **Payment** records.
5. A **Booking** may have zero or more **Check-In/Check-Out** records.
6. A **Booking** may generate zero or more **Notifications**.
7. A **Staff** member is a type of **User**.
8. An **Admin** manages Staff members and Users.
9. A **Staff** member may have one or more scheduled activities.
10. A **Schedule** may use one or more **Resources**.
11. A **Resource** may be maintained through the **Inventory**.
12. An **Admin** may generate zero or more **Reports**.
13. A **Report** contains information generated from the activities and records maintained by the system.

[5] Class Diagram.



Following are the code snippets that were written corresponding to each class and interface demonstrated in the above diagram-

PetOwner

```
public class PetOwner {
    private int owner_id;
    private String name;
    private String email;
    private String phone;
    private String address;

    public void register() {
    }

    public void login() {
```

```
    }

    public void updateProfile() {
    }
}
```

Pet

```
public class Pet {
    private int pet_id;
    private int owner_id;
    private String name;
    private String species;
    private String breed;
    private int age;
    private String special_needs;

    public void addPet() {
    }

    public void updatePet() {
    }

    public Pet getPetDetails() {
        return null;
    }
}
```

User

```
public class User {
    private int user_id;
    private String name;
    private String email;
    private String password;
    private String role;

    public void login() {
    }

    public void logout() {
    }
```

```
    public void updateProfile() {
    }
}
```

Staff

```
public class Staff extends User {
    private int staff_id;
    private String name;
    private String role;
    private String phone;
    private String email;
    private int schedule_id;

    public void viewSchedule() {
    }

    public void updateSchedule() {
    }
}
```

Admin

```
public class Admin {
    private int admin_id;
    private String name;
    private String email;
    private String password;

    public void generateReports() {
    }

    public void manageUsers() {
    }

    public void manageStaff() {
    }
}
```

BoardingService

```
public class BoardingService {
```

```
    private int booking_id;
    private int pet_id;
    private Date start_date;
    private Date end_date;
    private String status;
    private double total_cost;

    public void bookBoarding() {
    }

    public void updateBooking() {
    }

    public void cancelBooking() {
    }
}
```

GroomingService

```
public class GroomingService {
    private int booking_id;
    private int pet_id;
    private String service_type;
    private Date date;
    private String time;
    private String status;
    private double total_cost;

    public void bookGrooming() {
    }

    public void updateGrooming() {
    }

    public void cancelGrooming() {
    }
}
```

Booking

```
public class Booking {
    private int booking_id;
```

```
    private int pet_id;
    private Date start_date;
    private Date end_date;
    private String status;
    private double total_cost;

    public void bookBooking() {
    }

    public void updateBooking() {
    }

    public void cancelBooking() {
    }
}
```

Payment

```
public class Payment {
    private int payment_id;
    private int booking_id;
    private double amount;
    private String method;
    private String status;
    private String transaction_id;
    private Date payment_date;

    public void processPayment() {
    }

    public void refund() {
    }

    public void getPaymentDetails() {
    }
}
```

CheckInOut

```
public class CheckInOut {
    private int record_id;
    private int pet_id;
```

```
    private int booking_id;
    private Date check_in_datetime;
    private Date check_out_datetime;
    private String notes;

    public void checkIn() {
    }

    public void checkOut() {
    }

    public CheckInOut getRecord() {
        return null;
    }
}
```

Notification

```
public class Notification {
    private int notification_id;
    private int user_id;
    private String type;
    private String message;
    private Date sent_at;
    private String status;

    public void send() {
    }

    public void getNotification() {
    }
}
```

Schedule

```
public class Schedule {
    private int schedule_id;
    private int staff_id;
    private Date date;
    private String start_time;
    private String end_time;
```

```
    public void createSchedule() {
    }

    public void updateSchedule() {
    }
}
```

Resource

```
public class Resource {
    private int resource_id;
    private String resource_name;
    private String type;
    private int capacity;
    private boolean availability;

    public void updateResource() {
    }

    public boolean getAvailability() {
        return availability;
    }
}
```

Inventory

```
public class Inventory {
    private int inventory_id;
    private String item_name;
    private int quantity;
    private Date last_updated;

    public void addItem() {
    }

    public void updateItem() {
    }

    public List<Inventory> getLowStock() {
        return null;
    }
}
```

Report

```
public class Report {
    private int report_id;
    private String report_type;
    private Date period_from;
    private Date period_to;
    private Date generated_on;

    public void generate() {
    }

    public void export() {
    }

    public Report getReportDetails() {
        return null;
    }
}
```

Conclusion:

By performing this experiment, we were able to understand various Class Diagram related concepts such as **classes, attributes, methods, associations, generalization and multiplicity** in designing a software system. We were also able to identify the major entities and relationships involved in the proposed **Pet Boarding and Grooming Management System** and construct a Class Diagram representing the overall structure of the system. Finally, we were able to write the corresponding Java class declarations based on the designed Class Diagram.